

11-768 HW2: Evaluating a Data-Visualization Agent

Yubo Chen
Andrew ID: yuboc

Working-draft status. Parts 1–2 below record the completed baseline experiment, two evidence-backed error analyses, both proposed implementations, and the full seed-set comparison. Parts 3–4 are reserved for subsequent experiments. This is not yet the complete report for submission.

The four reproduced seed figures are unchanged copies in `report_assets/`. This directory must accompany the source to recompile the report; the PDF embeds the images.

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

Evaluation target and evidence. The object being evaluated is the validator’s ability to diagnose an existing visualization-agent run, not its ability to generate a new plot. The released `artifacts/<run_id>/result.json` files contain the task, input-file references, and agent trajectories; the corresponding `figure.png`, when present, is the final rendered artifact. The provided `seed_labels.json` supplies the human labels used for evaluation. The baseline’s predictions are stored separately in `output/baseline_predictions.json`. All comparisons are keyed by `run_id`, rather than relying on `task_id` being unique.

Baseline and serving configuration. The experiment used the supplied, unmodified `validator/baseline.py` and the fixed judge model `Qwen/Qwen3-VL-30B-A3B-Instruct-FP8`, served under the alias `validator` by `infrastructure/validator_model.py`. The supplied deployment uses one L40S GPU, a maximum of one serving container, and a 32,768-token maximum model context. The validator client uses temperature zero; each baseline judgment requests at most 250 output tokens. No alternative judge model was used.

For each run, the baseline first asks about execution failure. If that check returns an error, it stops evaluating the run. Otherwise, it separately asks about data correctness, chart design, and readability. Each question includes the task, serialized trajectory, input-file contents, and the figure if one exists. The installed baseline attempts full evidence first. On a recognized context-overflow error, its implementation compacts trajectory and input text and, if necessary, repeatedly downscales the image. This description follows the actual released implementation, rather than assuming that every request retains the original image resolution. The prediction file does not preserve the exact request used on each attempt, so I do not attribute any particular case below to truncation or downscaling.

Reproduction commands and coverage. The baseline was run and then scored using:

```
uv run python -m validator.runner \
  --solution validator.baseline \
  --artifacts artifacts \
  --out output/baseline_predictions.json

uv run python -m workflow score \
  --labels seed_labels.json \
  --predictions output/baseline_predictions.json
```

The saved predictions cover all 110 released runs, with 110 unique run IDs, no missing IDs, no unexpected IDs, and no duplicate entries. Scoring requires matching label and prediction coverage; the results below are not scores on an incomplete subset.

Scoring convention. Each error family is a separate binary classification problem. Here TP means correctly reporting an error of that family; TN means correctly not reporting it; FP is a false alarm; and FN is a missed error. A TN for one family does not imply that the run is free of errors in other families. The Matthews correlation coefficient is

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

The scorer reports zero when the denominator is zero. The macro-MCC is the unweighted average of the four family MCCs, calculated from their unrounded values; it is not an accuracy percentage.

There are 12 human-labeled execution failures. These runs are excluded from the other three families’ confusion counts, as required by the assignment. Consequently, execution failure is scored on 110 runs and the other families on 98. A *predicted* execution failure does not receive this exemption. Every family has both positive and negative ground-truth support in this set.

Error family	Scored runs	TP	TN	FP	FN	MCC
<code>execution_failure</code>	110	4	65	33	8	−0.0022
<code>wrong_data</code>	98	0	73	0	25	0.0000
<code>wrong_chart</code>	98	3	27	0	68	0.1096
<code>hard_to_read</code>	98	0	59	0	39	0.0000

Table 1: Measured baseline results against the released human labels. Macro-MCC is **0.0268**. Values are rounded to four decimal places.

Where disagreements concentrate. The baseline over-reports execution failure: it reports this family on 37 runs, of which only four are true positives and 33 are false positives. It also misses eight of the 12 actual execution failures. This yields an execution MCC very close to zero. On the 98 runs eligible for the remaining families, it detects none of the 25 data errors or 39 readability errors, and only three of 71 chart-design errors. The zero MCCs for data and readability reflect constant negative predictions, not zero accuracy: their TN counts are nonzero, but none of their positive examples are detected.

The execution-first control flow contributes to downstream omissions: among the 33 falsely predicted execution failures, the human labels contain three `wrong_data` positives, 14 `wrong_chart` positives, and eight `hard_to_read` positives. These counts overlap across runs and must not be summed as a number of distinct runs. They also do not explain all downstream false negatives.

In particular, the second error type below occurs on runs that were not stopped by the execution check.

Case-analysis method and limitations. For each selected example, the analysis compares the task requirements, the saved baseline prediction, the human label, the full trajectory (including tool calls and tool results), and the supplied final PNG. All four case-study images were opened, decoded, and visually inspected. The plotting scripts were not re-executed or used to replace these images. Agent-written claims of success are treated as claims to verify, not as independent ground truth. The saved prediction file retains explanations for reported errors, but not the raw responses for negative judgments; therefore, an empty error list alone cannot reveal the model’s reasoning.

1.2 Validator Error Type 1

Recurring pattern: success/failure polarity mismatch. Runs **022** and **079** are both assigned `execution_failure`, even though their saved explanations say that the agent executed successfully and produced the final figure. Run 079 explicitly states that **YES** is being used to indicate successful completion. This conflicts with the meaning of **YES** in the baseline’s execution-failure question. These two independently inspected runs establish recurrence; this analysis does not assume that all 33 execution false positives share the same cause.

Run 022: an acceptable artifact classified as failed execution. The task asks for two genuinely separate, side-by-side subplots showing the same monthly active-user series: a linear y-axis on the left and a logarithmic y-axis on the right. Both axes in each panel must be labeled, and the titles must identify the scales. The trajectory records `python create_figure.py` returning zero, followed by `ls -l figure.png` returning zero and listing a 42,528-byte file. The final status is **Submitted**. The supplied 1200-by-500 PNG is readable and visibly contains the two separate linear/log panels (Figure 1). The human label is an empty error list.

Nevertheless, the baseline labels the run `execution_failure`. Its explanation begins *The agent successfully completed the task* and also states that there was no crash, exhaustion of turns, or premature abandonment. The execution-failure label is therefore inconsistent with both the observable final artifact and the baseline’s own explanation. This is an execution-family FP, not evidence that the agent failed to deliver a plot.

Run 079: completed rendering with a different, genuine error. The task requests a filled 3D tricontour plot, with a radius range extending to 1.2, the CMRmap colormap, and an adjusted viewing angle. Both executions of `python generate_plot.py` return zero. The first emits a Matplotlib deprecation warning, rather than a terminating exception; the revised script runs with no reported output. The code saves `figure.png`, the run ends as **Submitted**, and the supplied 2340-by-1971 PNG can be decoded.

Importantly, producing this image does *not* mean that the task was fully satisfied. The final plotting call is a `contourf` projection with `zdir='z'` and a fixed `offset=np.min(Z)`. The supplied image shows contour bands on a flat plane at the bottom of the 3D box, rather than the requested filled 3D tricontour structure (Figure 2). The human label is therefore **wrong_chart**, not `execution_failure`.

The baseline instead reports execution failure. Its explanation says that the script executed without errors and that the figure was saved, then explicitly states that the answer is **YES** to indicate that the agent finished successfully. In particular, it says *the answer is YES to indicate that the agent finished the task successfully*. The explanation’s claim that the requested chart was

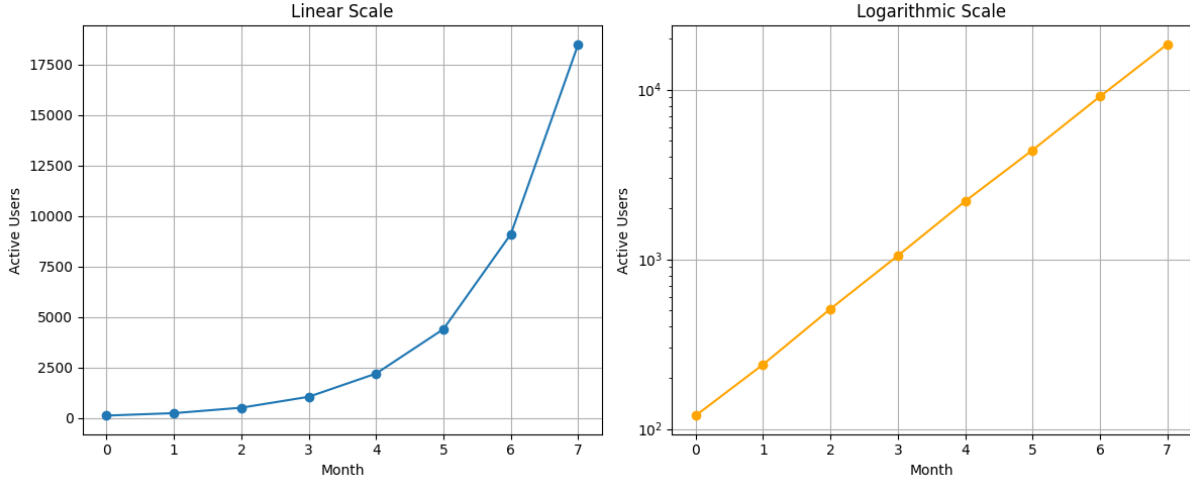


Figure 1: The supplied final artifact for run 022. The separate linear and logarithmic panels are present; the human label is acceptable.

fully correct should not be adopted as a fact: the final artifact and human label show otherwise. For this run, the baseline incurs an execution FP and a chart FN.

Implementation-level explanation and downstream consequence. In `validator/baseline.py`, `judge_execution` asks whether the agent crashed, exhausted its turns, or gave up without producing a valid figure. The intended answer for successful rendering is NO. However, `_ask` accepts any answer beginning with YES as a positive for the current error family; it does not check whether the explanation supports the label. The saved output has already removed the leading answer token, so I do not claim to have a complete raw-response log. Run 079’s retained explanation nevertheless explicitly documents the competing interpretation of YES as a success signal. The relevant source locations are `validator/baseline.py`, lines 158–168, for answer parsing and the execution question, and lines 194–198 for the early-return gate.

Finally, `validate` returns immediately after an execution error. Thus the incorrect execution decision on 079 suppresses its later chart check. Correcting the gate would restore the opportunity to evaluate the chart; it would not by itself guarantee that the chart error is detected. The evidence supports a mismatch between question polarity, response meaning, and parsing, rather than a claim that the code reverses every YES/NO answer or that this mechanism explains every baseline error.

1.3 Validator Error Type 2

Recurring pattern: explicit axis-label omissions accepted. Runs 021 and 202 omit axis titles explicitly required by their tasks, yet the baseline returns `errors: []` for both. Their human labels contain `wrong_chart`. Neither run is predicted as an execution failure, so this pattern is distinct from the premature execution gate in Section 1.2. The common error is a false negative on a concrete design requirement, not a subjective preference about appearance.

Run 021: tick labels mistaken for a labeled axis in the agent’s self-check. The task specifies a vertical box plot for the backend services `auth`, `catalog`, `search`, and `checkout`, with the service names as x-tick labels, and separately requires the chart to *label both axes*. The final

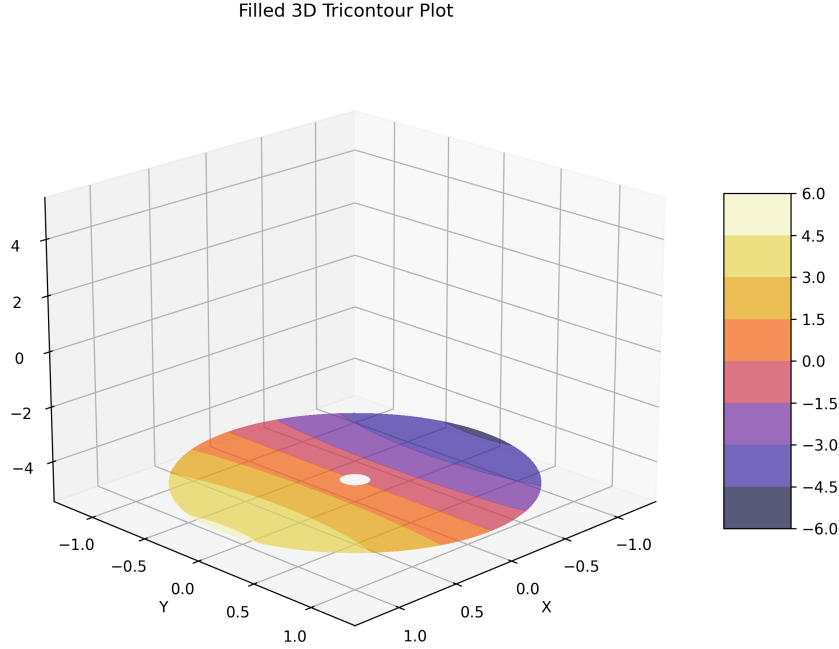


Figure 2: Run 079 delivered a readable PNG, but the contours lie on a fixed z -plane. This supports the provided `wrong_chart` label, not the baseline’s `execution_failure` label.

image has the y -axis title `Latency (ms)` and the four service-name tick labels, but no x -axis title (Figure 3). The last successfully executed version of `plot_latency.py` sets the y -label and title and rotates the ticks, but does not add an x -label. Its final execution returns zero, and the stored PNG is valid. Intermediate code errors were repaired; they are not evidence of a final execution failure.

The human label identifies the missing x -label as `wrong_chart`, whereas the baseline returns an empty error list. The agent’s final self-check lists the service names as `X axis labels`, illustrating the distinction that must be verified: tick labels identify individual categories, whereas an axis title identifies the axis. The task did not prescribe a particular x -axis-title string, so the violation is absence of a title, not failure to use an invented exact phrase.

Run 202: required labels lost in the final script. The task asks for standard and notched box plots in a 1-by-2 layout, with the x -axis label `Four separate samples`, the y -axis label `Measured values`, and horizontal gridlines in both panels. The final image retains the x -labels, but neither panel displays the required y -label; neither displays the requested gridlines (Figure 4). The human label records these two chart-design omissions, but the baseline again returns an empty error list.

The trajectory is particularly informative. An earlier `generate_boxplots.py` includes the y -labels and gridlines. Later, the agent writes a new `plot.py`; that final script retains `set_xlabel` but omits `set_ylabel` and the grid calls. It then successfully runs `python plot.py`, saves to `figure.png`, and submits. The last script and the supplied final image therefore corroborate the omissions. An earlier successful script cannot establish compliance after a later script has regenerated the output.

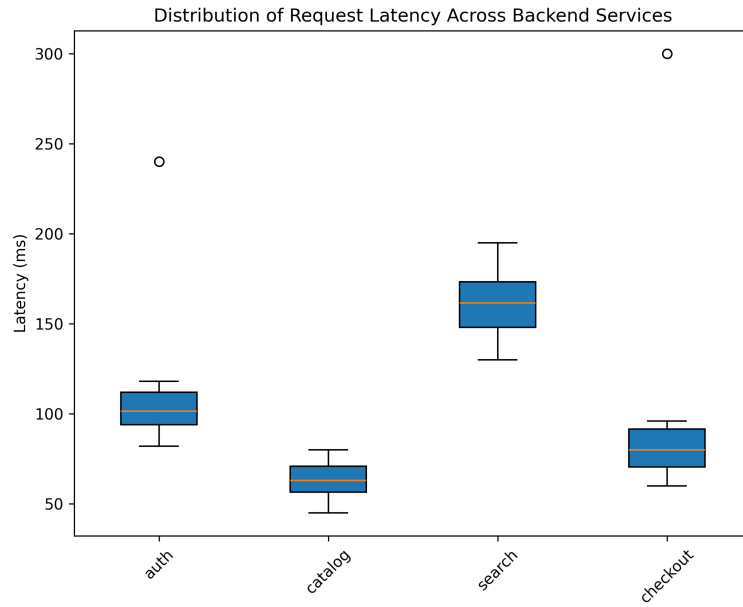


Figure 3: Run 021 has service-name tick labels and a y-axis title, but no x-axis title despite the explicit requirement to label both axes.

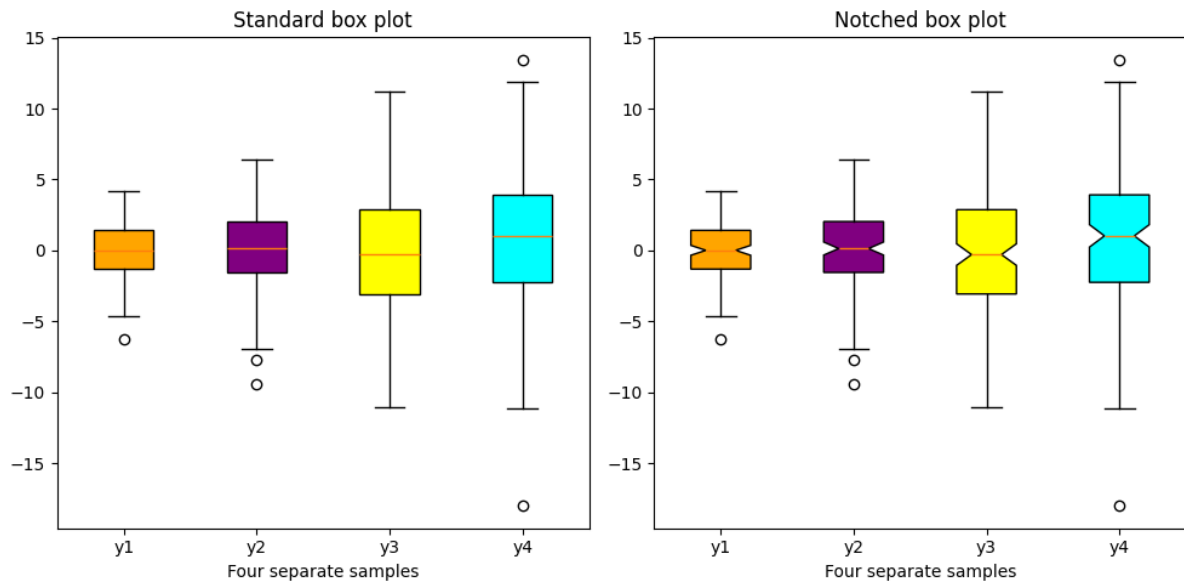


Figure 4: Run 202 retains the x-axis labels, but the specified **Measured values** y-axis labels and horizontal grids are absent from both final panels.

Why a different check is needed. The baseline asks a single broad question about whether the figure follows the requested chart design. It does not require an explicit audit of each requirement or identification of the final output-producing script. The two examples demonstrate that this procedure failed to detect clear axis-label omissions even when a figure was available.

Possible contributing factors are confusing tick labels with axis titles, trusting the agent’s self-description, or treating earlier correct code as evidence about the final figure. These are hypotheses motivated by the observed trajectories, not claims about the judge’s hidden reasoning. In particular, the empty predictions do not preserve the negative responses, so it would be unsupported to quote an invented model explanation for either run.

1.4 Proposed Validator Changes

The following changes were *proposed* after the baseline analysis. This section records their motivation before implementation; the incremental implementation and validation status is recorded separately in Part 2. No improved-validator scores are claimed in Part 1.

Change 1: artifact-grounded execution decisions with explicit failure polarity. Replace the bare YES/NO execution interface with an explicit internal decision field, such as `execution_failure: true/false`, plus specific supporting evidence. Define `true` only as failure to deliver a valid final figure, and `false` as permission to continue the remaining three checks, not as an assertion that the whole task is correct. Strictly validate the response schema and Boolean type rather than relying on a leading word or converting arbitrary strings to truth values.

Ground the decision in the actual final artifact and trajectory: check whether the required image exists and can be decoded, and examine the final relevant tool results and output path. These filesystem and decoding checks are deterministic evidence; the interpretation of the trajectory and rendered content can remain model-based. A decodable PNG alone is not proof of correct data, correct design, or readability. Likewise, `Submitted` and an agent-written success claim are not sufficient proof, and a repaired intermediate exception does not establish final failure.

A malformed or contradictory judge answer should trigger bounded rechecking or retry. If a reliable judgment cannot be obtained, raise a validator-side error so the existing runner can retry, rather than silently treating the run as acceptable or labeling the visualization agent as an execution failure. Do not invert labels merely because an explanation contains a word such as *success*. When execution really fails, retain the terminal-family convention; otherwise, continue all three nonterminal checks. This targets the documented polarity mismatch in 022 and 079 while preserving the distinction between delivery and task compliance.

Change 2: requirement-by-requirement audit of the final chart. Extract the task’s explicit chart-design requirements into a checklist, including their scope (which panel, axis, or whole figure they apply to). For each requirement, compare the final rendered figure with the last relevant output-producing code and tool results, recording a status such as pass, fail, or uncertain and concrete evidence. Explicitly distinguish axis titles from tick labels and verify exact text only when the task specifies exact text. Do not impose an additional x-label phrase on 021; do check `Measured values` on 020.

The final image is the evidence of what was actually rendered. Trace the full tool-call sequence to identify later overwrites rather than searching the entire trajectory for any earlier occurrence of a correct plotting call. Code inspection is corroboration, not a universal static rule: labels may be produced by helper functions, shared figure labels, or other valid plotting APIs. Whether a shared label is sufficient depends on the task, not on an unconditional requirement to repeat every label

on every panel. The agent’s statement that requirements are satisfied is not a substitute for this audit.

Uncertain items should receive targeted additional inspection with the same fixed judge model, rather than automatically passing. Confirmed design violations should be consolidated into one `wrong_chart` error with specific evidence; for 202, that evidence can mention both the missing y-label and gridlines. This keeps category boundaries intact: missing required labels are chart errors, not execution failure, and missing text is not automatically a readability error. Formatting or request failures must not be hidden as ordinary uncertainty and then converted into an empty error list. This proposal specifically targets the axis-label false negatives in 021 and 202, while the checklist mechanism generalizes to other explicitly requested design elements.

Constraints and validation plan. Both changes are restricted to the student validator while preserving `validate(run)` and the required list-of-errors output schema. The internal decision/checklist protocol does not replace the external `family/evidence` contract. All model-based steps will use the fixed Qwen3-VL judge; no other model will be called. The validator will operate on the current run’s evidence, without loading `seed_labels.json`, using run-ID lookup tables, or consulting other candidate runs to choose a prediction. The original baseline and its prediction file will be preserved as the comparison point.

Part 2 will test whether the changes reduce the targeted errors on the four case-study runs and on the complete seed set. It must also check for regressions: check and report whether execution false negatives increase because the new check is too permissive, or whether the checklist invents requirements or rejects permitted shared labels. All four family MCCs and macro-MCC will be compared with Table 1, alongside at least one remaining FP and FN. Structured output can still contain an incorrect semantic judgment, and a stricter evidence threshold can increase omissions; therefore, neither proposal is presented as a guaranteed improvement or as a demonstrated solution to the private grading set.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

Completed Part 2 scope. Both proposed changes have been implemented in `validator/solution.py`. The original `validator/baseline.py` and `output/baseline_predictions.json` remain unchanged. The execution gate and chart-design check are replaced; the data and readability checks retain the baseline procedures. The implementation is runnable and has been evaluated on all 110 released seed runs. The incremental tests below document development checks; the full results in Section 2.2 are the primary performance comparison.

Change 1: deterministic artifact checks. The execution gate first reads the actual delivered image. No recorded image path, a missing file, a directory in place of the file, an invalid PNG format, or a corrupt PNG yields a terminal `execution_failure` with specific artifact evidence. A present image must pass both Pillow’s structural `verify()` check and a fresh open with `load()` to decode pixels. The implementation does not search for an alternative output path or execute the agent’s plotting code to repair the delivered artifact. Permission and file-reading environment errors propagate to the runner rather than becoming an agent-failure label. Thus a successful shell command or an agent-written completion claim cannot compensate for a missing required output.

Change 1: model-based delivery judgment and strict parsing. For a decodable PNG, the same fixed Qwen3-VL model receives the task, input-file contents, full serialized trajectory including tool calls and tool results, and the image. The instructions separate final delivery from data correctness, chart design, and readability, and explicitly disregard repaired intermediate exceptions as evidence of final failure. A blank-looking image alone is not enough to diagnose execution failure: invisible plotted elements may instead be a readability problem. For a positive judgment about a decodable artifact, the prompt requires image and trajectory evidence that it is only a placeholder or an unrelated non-figure artifact; this requirement is not a programmatic proof of the model’s evidence.

The internal response has exactly three fields: `execution_failure`, `artifact_status`, and `evidence`. Only a real JSON Boolean is accepted. The status must be `rendered_figure` when failure is false, or `no_figure_content` when failure is true. Unknown or missing fields, duplicate keys, blank evidence, malformed JSON, and responses truncated at the output limit are rejected. A single enclosing JSON code fence is tolerated, but explanatory text outside the JSON is not. The public return type remains `list[Error]`, with the unchanged `family/evidence` schema.

There are at most two decision attempts for a decoded artifact, excluding transport retries and context-overflow handling. A valid false decision permits the remaining three family checks. A provisional true decision is rechecked against the category boundary and must be confirmed before returning an execution error. If the attempt budget expires without a reliable decision, the function raises a validator-side error for the runner, rather than guessing an agent label or returning an empty error list. The parser checks consistency between the structured fields; it cannot prove that all natural-language evidence is semantically consistent or that the model’s judgment is correct. The second judgment is a targeted recheck, not an independent ground-truth oracle.

Context handling and scope. The model client retains temperature zero, and execution judgments allow 512 output tokens. Full evidence is attempted first. Only a recognized context overflow triggers compaction using the server’s reported context limit and tokenizer. The compacted text budget reserves half the context for the image and formatting and explicitly reserves the execution

response budget. Input text may retain only its beginning and end. Trajectory compaction retains a chronological suffix of complete messages, normally considering up to the last 12, and preserves the last tool call/result group whose IDs can be linked. It does not retain a known tool result without its caller. If even the required final group cannot fit, the context error is surfaced. Earlier events, including some output-producing code, can still be omitted; this is not a guarantee that all relevant history survives. Omissions are disclosed to the judge and must not be interpreted as agent errors. If necessary, the existing baseline helper downscales the request image in memory; the supplied on-disk figure is never replaced.

All decisions use only the current run. The submitted validator does not load human labels or branch on public run IDs. The selected IDs below belong only to the separate test harness. The Step 2.1 experiment below predates the separate design-audit implementation and concerns only the execution gate; its recorded `implementation_sha256` identifies that earlier version, not the current two-change source.

Incremental local verification. The offline regression suite in `tests/test_solution_execution.py` contains 33 passing tests, with model and tokenizer responses mocked. It covers strict parsing, bounded rechecking, PNG validation, repaired exceptions, propagation of validator-side failures, terminal early return, and preservation of the nonterminal output schema. Context tests include oversized early history and a final tool group followed by completion messages; if only the completion message fits, the check must raise rather than silently drop the final known tool group. These are tests of program behavior, not evidence that the real judge always classifies synthetic blank or invisible charts correctly.

The read-only artifact audit in `tests/check_execution_smoke.py` inspected all 110 released runs: 12 have no delivered `figure.png`, and 98 PNGs pass structural verification and full decoding. Comparison with the provided labels confirms that those 12 missing outputs are the 12 human-labeled execution failures. This comparison is performed for offline evaluation only; labels are not input to the validator. The audit does not measure the model-based gate’s predictions on the 98 images and must not be interpreted as a full-validator MCC result.

The local checks can be reproduced without model requests:

```
uv run python -m unittest discover -s tests \
    -p test_solution_execution.py
uv run python tests/check_execution_smoke.py
```

Limited real-model smoke test. The execution gate alone was then tested on six selected released runs. The saved diagnostic record is `tests/diagnostics/step2_1_execution_checks.json`; it explicitly identifies its scope as execution-only and stores the actual judge responses and token usage. This is not the full prediction file required for Part 2. The test can be reproduced with the existing validator endpoint:

```
uv run python tests/check_execution_smoke.py \
    --live --run-ids 022 079 021 202 089 105
```

The five runs with PNGs each obtained a valid false execution decision on the first model call. Run 089 was rejected locally for its missing required output, without a model call. The five completed judge calls used 43,734 total tokens as reported by the service, including 751 completion tokens. No context-overflow or response-repair path was needed in this selected test.

The targeted execution false positives on 022 and 079 are absent in this smoke test. Run 105 adds another repaired-exception example: the baseline reports execution failure even though

Run	Human	Baseline	Step 2.1	Model calls
022	No	Yes	No	1
079	No	Yes	No	1
021	No	No	No	1
202	No	No	No	1
089	Yes	No	Yes	0
105	No	Yes	No	1

Table 2: Execution-failure decisions only: Yes means this family is present, not that the task is successful. All six selected execution decisions agree with the human labels. These are targeted examples, not an unbiased performance estimate or a complete four-family evaluation.

the final plot was delivered; the new gate permits further checks. Conversely, run 089 illustrates a corrected execution false negative. Its trajectory records successful plotting code that saves `plot.png`, but the required `figure.png` was not delivered. The new deterministic check diagnoses the actual missing output instead of adopting the completion claim.

The smoke test also exposes a remaining limitation rather than proving all judge reasoning reliable. For 079, the model’s internal false-decision explanation calls the chart a filled 3D tricontour, despite the flat contour projection discussed in Part 1. For 202, it incorrectly claims that the final chart has the required y-axis label and horizontal gridlines. Those details conflict with the supplied image and final plotting code. The delivery decisions are nevertheless correct: both runs have figures. Their internal explanations must not be reused as evidence that the design requirements pass. This motivates keeping the separate design audit in Change 2, rather than allowing a negative execution decision to certify the entire task. Passing the execution gate on 021 and 202 does not resolve their chart-design false negatives; Change 2 is tested separately below.

Change 2: task-only, scoped requirement extraction. The fixed judge first receives only the original task instructions and their numbered nonempty lines, not the candidate’s image, trajectory, or completion claims. The policy instructs it to extract individually checkable, explicit requirements for chart type/projection, layout, axes, legend/colorbar/annotations, and style. Each item records a unique ID, kind, requirement, affected panel/axis scope, and source-line ID. The implementation restores that line’s verbatim text as the item’s source quotation. The parser rejects unknown source IDs, extra/missing fields, invalid kinds, duplicate requirement IDs, and empty/non-string fields. A response using a source quotation instead of an ID is also accepted only when it passes the original strict quotation checks; it cannot bypass grounding. This prevents successful ID-based responses from rewriting quotations, but does not prove that their interpretations are supported by those lines or that extraction is complete. An omitted requirement, wrongly inferred scope, or an unsplit compound item remains a possible source of false negatives: atomic splitting is requested in the policy, not proved by the parser.

The extraction policy prohibits inventing unrequested labels, grids, legends, exact wording, or dimensions. It separates numerical correctness from design requirements. An axis label is not interchangeable with category tick names. Panel scope matters: a requirement for both panels cannot be satisfied by checking only one. An empty checklist is allowed for a task with no explicit design requirements; this too relies on the model’s extraction judgment.

Change 2: unprimed observation and final-artifact audit. Before an audit, a separate call receives only the final image, without task text, expected labels, input contents, or agent claims.

It describes visible panel positions, titles, separate axis labels versus tick labels, grids, annotations, and geometry. This is a fallible observation by the same fixed model, not a reference figure or an independent oracle. Its purpose is to reduce substitution of expected elements for observed ones.

The audit then receives that observation, the checklist, original task, input contents, full serialized trajectory, and decoded final PNG. The policy makes visible final image evidence primary and uses the latest successfully executed output-producing code to corroborate metadata and ambiguities. Correct earlier code and agent success claims do not certify the final artifact. This directly addresses the overwritten-script pattern in 202. Equivalent implementations, including pyplot functions, helpers, and shared/global labels when allowed by the task, are accepted; the validator does not decide label absence by searching for one API name.

Each checklist ID must appear exactly once in the audit with one of **pass**, **fail**, or **uncertain**, plus nonempty evidence. Missing, duplicate, or unknown IDs are rejected, not silently counted as passing. Only uncertain items receive one targeted recheck round. Confirmed failures are consolidated into one **wrong_chart** error, whose evidence includes each violated requirement, scope, task quote, and final-state observation. If no failure is established but an item remains uncertain, the check raises a validator-side error instead of claiming success. When a confirmed failure already establishes this binary family, it remains reportable even if another requirement is unresolved; the unresolved item is not presented as a demonstrated violation.

Both extraction and each audit allow at most two response-format attempts. Truncation, malformed JSON, invalid fields, and incomplete coverage trigger schema repair; exhausting the budget propagates an error. Transport and unavailable-context failures do not become agent labels. Extraction allows 3,072 output tokens, image-only observation 1,024, and audit 4,096. Observation also rejects empty/non-textual or truncated responses with a bounded retry; non-context service errors propagate. The audit’s context fallback budgets the full task, checklist, stage policy, and actual response allowance, rather than reusing the execution check’s 512-token response reservation. The task and checklist are never excerpted. Trajectory/input omissions and possible in-memory image downscaling retain the limitations described above.

To reduce persistent formatting failures observed during development, extraction and audit also request JSON-schema-constrained decoding through the existing fixed-model endpoint, following the vLLM structured-output interface. The extraction schema limits source IDs and kinds to their current-task allowed values. The audit schema allows only ID, status, and evidence, restricts IDs to the supplied checklist, and requires its number of entries. Local parsers still verify nonempty fields, unique IDs, and exact coverage. Format constraints do not prove correct interpretation, complete extraction, truthful visual evidence, or correct error-family attribution. No server deployment, model, runner, or external prediction schema was changed.

Data correctness still uses the baseline’s independent data-only question. The old combined data-and-chart helper is not called, preventing duplicate or contradictory chart judgments. Data and design errors can coexist. Readability remains a separate baseline check. For a delivered chart without repairs or uncertainty, this version normally makes six calls: execution, data, requirement extraction, image-only observation, design audit, and readability. The additional calls are a cost of explicit coverage, not evidence that the underlying fixed model is infallible.

Change 2: offline regression tests. The combined suite now has 91 passing tests: the 33 execution tests above and 58 design tests in `tests/test_solution_design.py`. Design tests cover grounded quotations, strict schemas and exact checklist coverage, bounded format repair, rejection of truncated answers, independent data/design labels, uncertain-item rechecking, and unresolved-error propagation. Mocked interface tests verify that extraction receives only task instructions,

whereas audit receives exact final-image bytes, raw tool calls/results, and input contents. Context tests verify the design-specific policy and response budget, checklist preservation, non-context error propagation, and image downscaling without changing the on-disk figure. Additional tests verify task-line ID restoration, image-only observation isolation and bounded retries, JSON-schema request parameters, targeted subset schemas, and unchanged format constraints during context fallback. A mocked shared-label case checks control-flow compatibility with an allowed global label; it is not a real-model generalization result.

```
uv run python -m unittest discover -s tests \
  -p 'test_solution_*.py'
```

Targeted development checks and limitations. The design-only harness `tests/check_design_smoke.py` was run on seven selected seed cases, not on the entire seed set. Its scope deliberately excludes execution, data, and readability classification. Early development checks exposed missed design requirements, rewritten source quotations, and audits that repeatedly copied extra checklist fields. These motivated source-line IDs and schema-constrained decoding. Such validator-side operational failures were not counted as negative labels. The superseded debug records are not used for the full-set results below.

The final selected test is saved in `tests/diagnostics/step2_2_design_checks.json`. Its recorded source hash matches the tested implementation. It completed all seven cases using 21 model calls, with 94,815 service-reported total tokens, including 9,183 completion tokens. Each case used one extraction, one image-only observation, and one audit; no format repair, uncertainty recheck, or context fallback was needed in this final selected test. The command is:

```
uv run python tests/check_design_smoke.py \
  --live --run-ids 021 202 022 105 079 097 196
```

Run	Human wrong-chart	Baseline wrong-chart	Step 2.2 wrong-chart
021	Yes	No	Yes
202	Yes	No	Yes
022	No	No	No
105	No	No	No
079	Yes	No	No
097	No	No	No
196	No	No	No

Table 3: Design-family membership only on selected cases. Baseline 079 and 105 instead return execution errors; No in their design column does not imply overall task success. These targeted observations are not a full-validator MCC estimate.

The final selected smoke-test labels recover the targeted design positives on 021 and 202, and all four selected design-negative controls remain negative. The controls exercise acceptable linear/log labeling (022), no unrequested figure-size constraint (105), pyplot-based labels and a plotted maximum marker (097), and a task that requires a y-axis label but not an x-axis label or a grid (196). These examples do not establish hidden-set generalization.

Critically, correct binary labels do not make every explanation correct. For 021 the new evidence correctly identifies the absent separate x-axis label, but it also wrongly insists that the valid label **Latency (ms)** must literally read **latency in milliseconds**. The task specifies the quantity and

unit, not that exact string. For 202 the final audit flags the grid requirement, but hallucinates vertical gridlines: the supplied image has no gridlines. It also incorrectly passes the missing **Measured values** labels. Image-only observations themselves sometimes hallucinate elements, and the later audit can repeat those claims or prefer earlier correct code. These failures remain despite valid schemas and grounded source IDs.

Run 079 remains a design false negative: the model accepts a flat contour projection on 3D axes as the requested filled 3D tricontour plot. The observation even describes both varying height and a single flat plane, illustrating unresolved semantic inconsistency. Consequently, this step supports limited improvement in family detection, not a claim that final code provenance, task interpretation, or image grounding is solved. The complete experiment below provides all four family scores and analyzes actual full-run false positives and false negatives. In particular, these selected smoke-test outcomes must not substitute for the full-run results.

2.2 Seed-Set Performance vs. Baseline

Experiment and coverage. The two-change validator was run on the entire released seed set, using the same fixed model and the unchanged public runner and prediction schema. The saved `output/pre_iteration_seed_predictions.json` contains exactly 110 predictions with 110 unique run IDs, matching the human-label IDs with no missing or additional runs. The official scorer validates the prediction schema. Each returned error has one of the four prescribed families and nonempty evidence. The reproduction commands are:

```
uv run python -m validator.runner \
  --artifacts artifacts \
  --out output/pre_iteration_seed_predictions.json

uv run python -m workflow.score \
  --labels seed_labels.json \
  --predictions output/pre_iteration_seed_predictions.json
```

The baseline comparison uses the separately preserved `output/baseline_predictions.json`; it is not regenerated with the new implementation. Labels are used only by offline scoring and analysis, never as validator inputs.

The unchanged Part 2 source at this checkpoint has SHA-256:

```
7b968ed3ccccbcc9b2fa6fb0f69f314453913ace8b0c41e1a4649495a0627551
```

This also matches the source hash in the final design-only diagnostic record. The full prediction JSON itself does not contain per-stage judge responses, token counts, timings, or a source-hash field; the hash above identifies the preserved implementation, not additional metadata embedded in that JSON. The Part 2 prediction file is retained separately from subsequent Part 4 predictions.

Interpretation and regressions. Macro-MCC increases from approximately 0.0268 to 0.2411. The execution gate eliminates all 33 baseline execution false positives and all eight execution false negatives on this seed set. The design family detects 41 of 71 positive runs, rather than the baseline's three, but also introduces five false positives. Thus the overall increase does not mean that every family improves: data detects only two of 25 positives with 15 false positives, while readability detects none of 39 positives with eight false positives. Both resulting MCCs are negative.

The unchanged data/readability procedures now run on candidates that the old execution gate rejected early. All 15 new data false positives and all eight new readability false positives lie

Metric	Baseline	Part 2
execution_failure	-0.0022	1.0000
wrong_data	0.0000	-0.1445
wrong_chart	0.1096	0.3512
hard_to_read	0.0000	-0.2424
Macro-MCC	0.0268	0.2411

Table 4: Full-seed comparison using the official scorer; MCC values are rounded to four decimal places. Higher is better, and negative values indicate negative correlation, not merely a low accuracy percentage.

Family	Evaluated runs	TP	TN	FP	FN
execution_failure	110	12	98	0	0
wrong_data	98	2	58	15	23
wrong_chart	98	41	22	5	30
hard_to_read	98	0	51	8	39

Table 5: Part 2 confusion counts. The 12 human-labeled execution failures are excluded from the other families, as required by the scoring rule; a predicted execution failure would not exempt a nonterminal gold run from evaluation.

among those 33 baseline execution false positives. This is an observed relationship between saved predictions, not proof that each model judgment was otherwise identical across experiments. It shows why correcting a terminal gate can expose downstream weaknesses. There is no ablation that separates the contributions of the gate and design audit; the reported gains belong to the complete two-change pipeline.

Runtime and context limitations. For an ordinary delivered chart, the new pipeline normally makes six model calls rather than the baseline’s four. The old baseline also stopped early on many valid charts, whereas Part 2 checks all three nonterminal families on those charts. Larger structured design responses, retries, model/container startup, and context fallback can add runtime. The prediction file does not record the full run’s actual call count, cost, or wall time, so the selected smoke-test token totals above are not extrapolated as measured full-run totals.

A context-overflow warning was observed during the full evaluation. The implementation attempts its documented compaction/downscaling fallback; the completed file has full coverage and valid schema. This establishes completion, not preservation of every original detail after fallback. The saved predictions do not identify the final retained context or resolution for each stage. Consequently, I do not attribute a particular false negative to that warning. Loss of earlier code, omitted input rows, or lower image resolution remain general risks of the fallback.

2.3 Qualitative Analysis of Targeted Error Types

Error type 1: execution polarity and final delivery. The full evaluation confirms the targeted gate improvement, not just the selected smoke test. Runs 022 and 079 are no longer labeled execution failures: both have decodable delivered plots, despite the baseline’s false-positive execution judgments. Run 089 now receives only the terminal execution label because its required output is missing, even though its trajectory contains successful code that saved a differently named file.

Together with the full confusion counts, these outcomes support mitigation of the Part 1 execution pattern on the released set. They do not establish perfect performance on unseen decodable placeholders or corrupted artifacts.

Removing an execution error does not guarantee overall correctness. Run 022 now has a spurious data label, and run 079 still has an undetected design error. The table summarizes the full predictions for the original four examples:

Run	Human families	Baseline families	Part 2 families
022	None	Execution	Data
079	Chart	Execution	None
021	Chart	None	None
202	Chart	None	Chart

Table 6: Full-validator outcomes on the Part 1 examples. Execution, Data, and Chart abbreviate the prescribed family names; None means an empty error list. The overall judgment on 022 is still incorrect even though its execution false positive was fixed.

Error type 2: explicit axis requirements and final-artifact grounding. Mitigation is partial. Run 202 changes from a design false negative to a family-level true positive, but run 021 remains a false negative in the full evaluation. For 202, the saved design evidence says the grids are vertical rather than horizontal. Inspection of the actual image shows no gridlines in either orientation; the required y-axis labels are also missing. Its correct binary family therefore does not demonstrate accurate enumeration or explanation of every violation. The combined two-change pipeline improves aggregate chart detection but does not solve visual hallucination, requirement omission, or preference for earlier correct code.

2.3.1 False positive: run 196, correct title reported as wrong

The task requests blue and orange groups, a beeswarm plot with box plots on the same chart, a y-axis label, a legend, and the exact title `Beeswarm plot and Boxplots, made with matplotlib`. The delivered PNG and final plotting code use that exact title, and the human label is an empty error list. The full Part 2 prediction nevertheless returns `wrong_chart` for the title. Its evidence repeats identical required and observed strings and ends with the statement `The title is correct`. There is no title mismatch to support this positive label.

This is a directly observable contradiction between a label and its evidence, not an ambiguous stylistic preference. The current parser enforces valid JSON, checklist IDs, statuses, and nonempty evidence, but does not establish that the evidence semantically entails a failure. A structurally valid failure can therefore become a false positive. The full file does not retain the original extraction and audit responses, so the precise hidden step that produced this decision cannot be reconstructed. A general evidence–decision consistency check, rather than a rule for this title or run ID, is one possible later improvement.

2.3.2 False negative: run 021, missing separate x-axis label

The task requires labeling both axes. As established in Part 1, the delivered PNG has service names as x-axis tick labels and a valid y-axis label `Latency (ms)`, but no separate x-axis label. The final executed plotting code contains the y-label and tick configuration without an x-label. The human label is `wrong_chart`, while the complete Part 2 prediction has `errors=[]`.

An omitted axis requirement, confusion between tick labels and an axis label, or an incorrect passing audit are plausible mechanisms; the saved full-run file contains no negative per-item audit log to distinguish them. The selected design-only smoke test detected this missing label, but that outcome does not override the full-run false negative. Run 196 likewise passed the selected smoke test and failed in the full run, despite the preserved source matching the smoke-test hash. These differences demonstrate that one selected successful check is insufficient evidence of repeatable correctness. They do not, by themselves, establish sampling randomness or context compaction as the cause. More reliable explicit-axis observation and semantic coverage checking are still missing.

Additional failure boundaries: data versus design, and 3D geometry. Run 022 illustrates a downstream regression. Its final code plots the same provided active-user values in both panels, using the required linear and logarithmic scales. The full prediction’s `wrong_data` explanation instead discusses titles, label formatting, and a preference that the two lines have the same color—a constraint not in the task. It identifies no incorrect value or selected subset. This points to leakage of design/aesthetic judgments into the unchanged data check, newly exposed when the execution gate permits it to run.

Run 079 remains a design false negative. The executed `contourf` uses `zdir='z'` and a fixed `offset=np.min(Z)`, and the delivered image shows a flat contour projection on 3D axes rather than the requested filled 3D tricontour geometry. The new full prediction is empty. The separate smoke audit’s contradictory descriptions of varying height and a single flat plane suggest weak geometric interpretation, but are not the full experiment’s hidden reasoning. Checking the presence of 3D axes alone cannot certify the required geometry.

Part 2 conclusion. Both Part 1 proposals are implemented, the complete seed experiment is saved, and all five required scores, targeted effects, regressions, and concrete FP/FN analyses are reported. Further tuning is deferred: the remaining semantic-consistency, coverage, family-boundary, and geometry weaknesses can inform the new-task experiments and subsequent iteration in Parts 3–4. No claim of perfect seed or private-set performance is made.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

Pending analysis of two visualization-task classes that may challenge the Part 2 validator.

3.2 Task Design

Pending at least five authored tasks, with at least two tasks in each of the two identified classes; document inputs, sources, and design rationale.

3.3 Agent Runs and Labeling

Pending one run per authored task for each of the three fixed agents, and human review of every resulting run.

3.4 Validator Performance on New Tasks

Pending the authored-run predictions and all four family MCCs plus macro-MCC; distinguish agent failures from validator mistakes.

3.5 Harbor Packaging

Pending actual Harbor evaluations: all authored reference solutions must score 1.0; two plausible-but-wrong mutants must demonstrate rejection and acceptance respectively. The final report must explain both rewards, the verifier’s boundaries for all four error families, and why a task-specific verifier cannot grade unknown private tasks.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

Pending at least one additional improvement motivated by observed Part 3 results.

4.2 Final Evaluation on Seed and New Tasks

Pending separate before/after comparisons on the seed and authored runs, reporting all four family MCCs and macro-MCC for each subset.

4.3 Qualitative Analysis of Self-Authored Tasks

Pending case-based analysis of improvements and regressions, including at least one FP and one FN with trajectory evidence and a hypothesis for the error.

4.4 Further Changes

Pending completion of the required experiments. Any further changes will be reported with their motivation and measured qualitative and quantitative effects; none are claimed in this draft.